



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA

Escuela Profesional de Ingeniería de Sistemas

Informe Final

Proyecto *Administrador de BD en consola o terminal*

Curso: Base de Datos II

Docente: Patrick Cuadros Quiroga

Integrantes:

Jahuira Pilco, Dayan Elvis (2022075749)

Mamani Cori, Cristhian Carlos (2023077282)

**Tacna – Perú
2026**

CONTROL DE VERSIONES					
Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	MPV	ELV	ARV	10/10/2020	Versión Original

INDICE GENERAL

1. Antecedentes	1
2. Planteamiento del Problema	4
a. Problema	
b. Justificación	
c. Alcance	
3. Objetivos	6
4. Marco Teórico	
5. Desarrollo de la Solución	9
a. Análisis de Factibilidad (técnico, económica, operativa, social, legal, ambiental)	
b. Tecnología de Desarrollo	
c. Metodología de implementación (Documento de VISION, SRS, SAD)	
6. Cronograma	11
7. Presupuesto	12
8. Conclusiones	13
Recomendaciones	14
Bibliografía	15
Anexos	16
Anexo 01 Informe de Factibilidad	
Anexo 02 Documento de Visión	
Anexo 03 Documento SRS	
Anexo 04 Documento SAD	
Anexo 05 Manuales y otros documentos	

Antecedentes

El presente informe final documenta el desarrollo completo del proyecto Administrador de BD en consola o terminal (NexusDB), elaborado en el marco del curso Base de Datos II. El proyecto se sustenta en cuatro entregables previos ya validados: el Informe de Factibilidad (FD01), el Documento de Visión (FD02), la Especificación de Requerimientos de Software (FD03) y el Documento de Arquitectura de Software (FD04), cuyos resultados se resumen y consolidan en este documento.

Durante el desarrollo, el alcance original del proyecto (un CLI relacional simple) se amplió progresivamente hasta incluir soporte NoSQL, un módulo de migración ETL, generación de consultas SQL asistida por IA, y tres canales de distribución: una extensión para Visual Studio Code, un servidor MCP (Model Context Protocol) y un bot de Telegram en modo de solo lectura.

Planteamiento del Problema

Problema

Las herramientas actuales de administración de bases de datos suelen abstraer los procesos internos mediante interfaces gráficas, lo que limita la comprensión técnica de los procesos de manipulación de datos. Además, en entornos profesionales y de servidores, donde predomina el uso de la línea de comandos, las herramientas existentes (psql, mysql, mongosh) presentan una curva de aprendizaje elevada para usuarios en formación.

Justificación

El desarrollo de una herramienta CLI unificada, didáctica y de código abierto permite a estudiantes y desarrolladores practicar la administración de bases de datos relacionales y NoSQL sin depender de licencias comerciales. Asimismo, la integración con protocolos emergentes de IA (MCP) y canales de distribución modernos (Marketplace, PyPI, Telegram) aporta valor formativo adicional al equipo desarrollador, en línea con las tendencias actuales de la industria.

Alcance

El sistema permite administrar bases de datos relacionales (PostgreSQL, MySQL, SQLite) y no relacionales (MongoDB, Redis, Cassandra) mediante una interfaz de consola, con operaciones CRUD, migración ETL entre motores, generación de SQL asistida por IA, panel de rendimiento, comparador de esquemas y gestión de usuarios. Se distribuye mediante una extensión de Visual Studio Code (Marketplace), un servidor MCP (PyPI) y un bot de Telegram de solo lectura. Queda fuera del alcance el desarrollo de un motor de base de datos propio y las operaciones de escritura desde las integraciones externas.

Objetivos

Objetivo General

Desarrollar una aplicación de consola que permita administrar bases de datos relacionales y no relacionales mediante comandos, distribuible a través de múltiples canales, incluyendo integraciones con asistentes de inteligencia artificial.

Objetivos Específicos

- Implementar la conexión y operaciones CRUD sobre seis motores de bases de datos (SQLite, PostgreSQL, MySQL, MongoDB, Redis, Cassandra).
- Desarrollar un módulo de migración ETL entre distintos motores de base de datos.
- Integrar un módulo de generación de consultas SQL a partir de lenguaje natural mediante IA.
- Distribuir el sistema mediante una extensión de Visual Studio Code, un servidor MCP y un bot de Telegram.
- Restringir las integraciones externas a operaciones de solo lectura, por motivos de seguridad.

Marco Teórico

Un CLI (Command Line Interface) es una interfaz basada en texto que permite operar un sistema mediante comandos escritos, en contraposición a una interfaz gráfica (GUI). Los sistemas gestores de bases de datos (SGBD) relacionales (PostgreSQL, MySQL, SQLite) organizan la información en tablas y usan SQL como lenguaje de consulta, mientras que los SGBD NoSQL (MongoDB, Redis, Cassandra) emplean modelos de datos alternativos (documentos, clave-valor, columnas anchas) orientados a escalabilidad y flexibilidad de esquema.

El Model Context Protocol (MCP) es un estándar abierto, publicado por Anthropic en 2024, que define cómo un asistente de inteligencia artificial puede invocar herramientas externas de manera estandarizada. A diferencia de una integración propietaria, un servidor MCP puede ser utilizado por cualquier cliente compatible (Claude, Cursor, Windsurf, Antigravity, entre otros). Un bot de Telegram, por su parte, es una aplicación que se ejecuta sobre la API de Bots de Telegram, permitiendo automatizar respuestas a comandos enviados por chat.

Desarrollo de la Solución

Análisis de Factibilidad

Según el Informe de Factibilidad (FD01), el proyecto resulta viable en todas sus dimensiones. La inversión total estimada asciende a S/. 18,420.00, con un Valor Actual Neto (VAN) de S/. 3,373.27, una Tasa Interna de Retorno (TIR) de 21.9% y una relación Beneficio/Costo de 1.18, todos favorables considerando una tasa de descuento del 12%. La factibilidad técnica se sustenta en el uso de tecnologías de código abierto y equipos ya disponibles por el equipo desarrollador; la operativa, en la baja curva de aprendizaje gracias al comando de ayuda integrado; la legal, en el uso exclusivo de licencias permisivas (MIT y de terceros); la social, en el valor didáctico de la herramienta; y la ambiental, en el bajo consumo de recursos al tratarse de software puro.

Tecnología de Desarrollo

El sistema fue desarrollado en Python 3.11, utilizando prompt_toolkit para el REPL y Rich para el formateo de tablas en consola. Para la conexión a bases de datos se emplearon psycopg2 (PostgreSQL), mysql-connector-python (MySQL), pymongo (MongoDB), redis (Redis) y cassandra-driver (Cassandra). La generación de SQL asistida por IA utiliza las APIs de OpenAI y Anthropic. El servidor MCP se construyó con el SDK oficial (mcp) y se distribuye como paquete Python en PyPI. El bot de Telegram utiliza python-telegram-bot y se despliega como servicio systemd en un VPS Debian 13. La extensión de Visual Studio Code se desarrolló en TypeScript, empaquetando el ejecutable de NexusDB (generado con PyInstaller) para su distribución en el Marketplace oficial.

Metodología de implementación (Documento de VISION, SRS, SAD)

El desarrollo siguió un enfoque iterativo e incremental a lo largo de 4 meses, partiendo del Documento de Visión (FD02) para definir el alcance, la Especificación de Requerimientos de Software (FD03) para detallar los requerimientos funcionales y no funcionales mediante casos de uso, y el Documento de Arquitectura de Software (FD04) para establecer las vistas 4+1 (lógica, implementación, procesos, despliegue y casos de uso) que guiaron la construcción del sistema. Cada incremento agregó una capacidad funcional completa (motor relacional, motor NoSQL, ETL, IA, extensión VSC, servidor MCP, bot de Telegram), validada mediante pruebas manuales antes de integrarse al resto del sistema.

Cronograma

Mes	Actividades
Mes 1	Análisis de factibilidad, documento de visión, CLI relacional (SQLite, PostgreSQL, MySQL) con operaciones CRUD.
Mes 2	Soporte NoSQL (MongoDB, Redis, Cassandra), módulo de migración ETL, especificación de requerimientos (SRS).
Mes 3	Generación de SQL con IA, arquitectura de software (SAD), extensión de Visual Studio Code y servidor MCP.
Mes 4	Bot de Telegram, despliegue en VPS, pruebas integrales de las tres integraciones, redacción del informe final.

Presupuesto

Categoría	Total (\$/.)
Costos Generales (equipos)	5 040
Costos Operativos (energía, internet, VPS, IA)	1 060
Costos del Ambiente (pruebas, repositorio)	320
Costos de Personal	12 000
TOTAL GENERAL	18 420

Conclusiones

- El proyecto NexusDB (Administrador de BD en consola o terminal) es una solución viable en las dimensiones técnica, económica, operativa, legal, social y ambiental, según lo demostrado en el Informe de Factibilidad.
- El sistema cumple los objetivos planteados: conecta y administra seis motores de bases de datos, ejecuta operaciones CRUD y ETL, genera SQL asistido por IA, y se distribuye mediante tres canales externos (VS Code Marketplace, PyPI y Telegram).
- Restringir las integraciones externas a operaciones de solo lectura resultó una decisión de seguridad determinante, evitando exponer las bases de datos conectadas a operaciones de escritura no supervisadas.
- El proyecto tiene un alto valor formativo, al haber expuesto al equipo desarrollador a tecnologías emergentes (protocolo MCP, agentes de IA) y a un ciclo real de distribución de software (empaquetado, publicación y despliegue en producción).

Recomendaciones

- Realizar pruebas con cada uno de los seis motores de base de datos soportados antes de la presentación final.
- Preparar un script de demostración que muestre la extensión de VS Code, el servidor MCP y el bot de Telegram funcionando en vivo.
- Rotar las credenciales utilizadas durante el desarrollo y las pruebas (VPS, bot de Telegram, bases de datos) antes de dejar el proyecto en un repositorio público.
- Para versiones futuras, evaluar la exposición de operaciones de escritura controladas (con confirmación explícita) en las integraciones externas.

Bibliografía

Ramakrishnan, R., & Gehrke, J. (2003). Sistemas de Gestión de Bases de Datos (3ª ed.). McGraw-Hill.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7ª ed.). McGraw-Hill Education.

Beaulieu, A. (2020). Learning SQL (3ª ed.). O'Reilly Media.

Python Software Foundation. (2026). The Python Standard Library.